Build Task — Versioned Release Gate for Project 5 — Hints & Solution

> Back to the Build Task · Hint 1 · Hint 2 · Solution

Only 2 hints — work through them in order, and don't jump ahead until you've genuinely tried. This Build Task pulls together everything from the 5 practice exercises above it — a version log, a gate, a live pointer that resists tampering, and a rollback that goes back to the last version *known to be good* — into one working system.

- Hint 1 — What you're building, and the exact pieces
- Hint 2 — The plan, and almost the whole thing
- Solution

> Back to the Build Task · Hint 1 · Hint 2 · Solution

Hint 1 — What you're building, and the exact pieces

Think of this Build Task as 3 real jobs, not 1: **storing** versions (`versioning.py`), **gating** a release (`deploy_gate.py`), and **recovering** from a bad one (`rollback.py`) — each file does one job, and each reads/writes the same underlying data, not 3 separate copies of the truth.

The design that makes "the gate can't be skipped by accident" actually true: instead of a `live_version.txt` that any text editor can rewrite with zero friction, keep an **append-only event log** — every `deploy()` call, pass or fail, appends one event; every `rollback_to_previous()` call appends one more. "What's live right now" is never stored directly — it's *derived*, every time, as "the version named in the most recent `deploy_pass` or `rollback` event." There's no separate pointer file left for someone to edit without realizing there's a gate behind it at all.

The exact pieces:

- `save_version(name: str, config: dict) -> VersionRecord` and `get_version(name: str) -> VersionRecord | None` in `versioning.py` — a thin, reusable layer other files build on, same shape as the `version_log` exercise.
- `deploy(version_name: str, threshold: float) -> DeployResult` in `deploy_gate.py` — looks up the version, runs `run_eval_suite(config)` , and appends a `deploy_pass` or

`deploy_fail` event to the history depending on the score, exactly like `release_gate`'s exercise, but recording an event instead of writing a pointer file.

- `get_live_version() -> str | None` and `get_live_version_at(timestamp: str) -> str | None` — both scan the same event history; the second just ignores every event after `timestamp`.
- `rollback_to_previous() -> VersionRecord` in `rollback.py` — finds the currently live version from the history, then walks backward through earlier `deploy_pass` / `rollback` events to find the most recent *different* version, and appends a `rollback` event pointing at it. This is exactly `rollback_trigger`'s idea of "go back to the last version *known to have passed*," now wired to real version records instead of a stub.
- `pytest` basics, as in Doc13: a test file imports the functions under test, and each `test_...` function sets up a small scenario and asserts the result. Give each test its own file names (and delete them at the start), so tests never share data with each other or with your real `versions.json`.

> Back to the Build Task · Hint 1 · Hint 2 · Solution

Hint 2 — The plan, and almost the whole thing

```
versioning.py:
    @dataclass VersionRecord: version_name: str, config: dict, created_at: str

    function load_versions(path) -> list[VersionRecord]
    function save_version(name, config, path) -> VersionRecord
    function get_version(name, path) -> VersionRecord | None

deploy_gate.py:
    @dataclass DeployResult: version_name: str, passed: bool,
                             score: float, failing_cases: list

    class VersionNotFoundError(Exception)

    function load_history(path) -> list[dict]
    function record_event(event_type, version_name, path) -> None
    function get_live_version(path) -> str | None
    function get_live_version_at(timestamp, path) -> str | None
    function deploy(version_name, threshold, versions_path,
                    history_path) -> DeployResult

rollback.py:
```

```
    class NoPreviousVersionError(Exception)
    function rollback_to_previous(history_path, versions_path) -> VersionRecord
```

```
# practice/build_task/versioning.py
import json
from dataclasses import dataclass, asdict
from datetime import datetime, timezone
from pathlib import Path

# why: one saved version as real fields, not a loose dict
@dataclass
class VersionRecord:
    version_name: str
    config: dict
    created_at: str

def load_versions(path: str = "versions.json") -> list[VersionRecord]:
    if not Path(path).exists():
        return []
    with open(path) as f:
        raw = json.load(f)
    records = []
    for item in raw:
        records.append(VersionRecord(
            version_name=item["version_name"],
            config=item["config"],
            created_at=item["created_at"],
        ))
    return records

def save_version(name: str, config: dict,
                 path: str = "versions.json") -> VersionRecord:
    # how: load first, then append — the version_log exercise's rule
    versions = load_versions(path)
    record = VersionRecord(
        version_name=name,
        config=config,
        created_at=datetime.now(timezone.utc).isoformat(),
    )
    versions.append(record)
    as_dicts = []
    for v in versions:
        as_dicts.append(asdict(v))
    with open(path, "w") as f:
        json.dump(as_dicts, f, indent=2)
    return record
```

```
def get_version(name: str,
                path: str = "versions.json") -> VersionRecord | None:
    for v in load_versions(path):
        if v.version_name == name:
            return v
    return None
```

```
# practice/build_task/deploy_gate.py
import json
from dataclasses import dataclass
from datetime import datetime, timezone
from pathlib import Path

from versioning import get_version

class VersionNotFoundError(Exception):
    pass

# why: the gate's decision as real fields — release_gate's Approach 2
@dataclass
class DeployResult:
    version_name: str
    passed: bool
    score: float
    failing_cases: list

def run_eval_suite(config: dict) -> dict:
    # stand-in for your real Doc13 suite — swap it in here
    if "politely" in config.get("prompt", ""):
        return {"score": 1.0, "failed": []}
    return {"score": 0.6, "failed": ["greeting_case", "citation_case"]}

def load_history(path: str = "history.json") -> list[dict]:
    if not Path(path).exists():
        return []
    with open(path) as f:
        return json.load(f)

def record_event(event_type: str, version_name: str,
                 path: str = "history.json") -> None:
    # why: append-only — every gate decision stays on record, with a
    # time, so "what was live at time T" can always be answered
    history = load_history(path)
    history.append({
        "event": event_type,
```

```
        "version_name": version_name,
        "at": datetime.now(timezone.utc).isoformat(),
    })
    with open(path, "w") as f:
        json.dump(history, f, indent=2)

def get_live_version_at(timestamp: str,
                        path: str = "history.json") -> str | None:
    # how: "live" is the latest pass or rollback up to that time —
    # worked out from the history, never stored in a separate file
    passing = []
    for event in load_history(path):
        counts = event["event"] in ("deploy_pass", "rollback")
        if counts and event["at"] <= timestamp:
            passing.append(event)
    if not passing:
        return None
    return passing[-1]["version_name"]

def get_live_version(path: str = "history.json") -> str | None:
    now = datetime.now(timezone.utc).isoformat()
    return get_live_version_at(now, path)

def deploy(version_name: str, threshold: float,
           versions_path: str = "versions.json",
           history_path: str = "history.json") -> DeployResult:
    record = get_version(version_name, versions_path)
    if record is None:
        raise VersionNotFoundError(f"No saved version named: {version_name}")

    result = run_eval_suite(record.config)
    passed = result["score"] >= threshold
    if passed:
        event_type = "deploy_pass"
    else:
        event_type = "deploy_fail"
    record_event(event_type, version_name, history_path)

    return DeployResult(
        version_name=version_name,
        passed=passed,
        score=result["score"],
        failing_cases=result["failed"],
    )
```

Now write `rollback.py`'s `rollback_to_previous()` yourself, using the same walk-backward-through-history idea from Hint 1, and compare against the Solution.

Back to the Build Task · Hint 1 · Hint 2 · Solution

Solution

Story — `versioning.py`: the version log from `version_log`, with each record as a typed `VersionRecord`. **If not:** every other file would reach into loose dicts and hope the key names matched.

```
# practice/build_task/versioning.py
import json
from dataclasses import dataclass, asdict
from datetime import datetime, timezone
from pathlib import Path

# why: one saved version as real fields, not a loose dict
@dataclass
class VersionRecord:
    version_name: str
    config: dict
    created_at: str

def load_versions(path: str = "versions.json") -> list[VersionRecord]:
    if not Path(path).exists():
        return []
    with open(path) as f:
        raw = json.load(f)
    records = []
    for item in raw:
        records.append(VersionRecord(
            version_name=item["version_name"],
            config=item["config"],
            created_at=item["created_at"],
        ))
    return records

def save_version(name: str, config: dict,
                 path: str = "versions.json") -> VersionRecord:
    # how: load first, then append — the version_log exercise's rule
    versions = load_versions(path)
    record = VersionRecord(
        version_name=name,
        config=config,
        created_at=datetime.now(timezone.utc).isoformat(),
    )
```

```
    versions.append(record)
    as_dicts = []
    for v in versions:
        as_dicts.append(asdict(v))
    with open(path, "w") as f:
        json.dump(as_dicts, f, indent=2)
    return record

def get_version(name: str,
                path: str = "versions.json") -> VersionRecord | None:
    for v in load_versions(path):
        if v.version_name == name:
            return v
    return None
```

Story — `deploy_gate.py`: `release_gate`'s gate with its `DeployResult` (Approach 2), plus `gate_bypass`'s idea taken one step further: every gate decision is a timestamped event in an append-only history, and "what's live" is worked out from that history — there is no pointer file to edit. **If not:** the Build Task's "not skippable by accident" constraint would rest on nobody touching a file.

```
# practice/build_task/deploy_gate.py
import json
from dataclasses import dataclass
from datetime import datetime, timezone
from pathlib import Path

from versioning import get_version

class VersionNotFoundError(Exception):
    pass

# why: the gate's decision as real fields — release_gate's Approach 2
@dataclass
class DeployResult:
    version_name: str
    passed: bool
    score: float
    failing_cases: list

def run_eval_suite(config: dict) -> dict:
    # stand-in for your real Doc13 suite — swap it in here
    if "politely" in config.get("prompt", ""):
        return {"score": 1.0, "failed": []}
    return {"score": 0.6, "failed": ["greeting_case", "citation_case"]}

def load_history(path: str = "history.json") -> list[dict]:
```

```
    if not Path(path).exists():
        return []
    with open(path) as f:
        return json.load(f)

def record_event(event_type: str, version_name: str,
                 path: str = "history.json") -> None:
    # why: append-only — every gate decision stays on record, with a
    # time, so "what was live at time T" can always be answered
    history = load_history(path)
    history.append({
        "event": event_type,
        "version_name": version_name,
        "at": datetime.now(timezone.utc).isoformat(),
    })
    with open(path, "w") as f:
        json.dump(history, f, indent=2)

def get_live_version_at(timestamp: str,
                        path: str = "history.json") -> str | None:
    # how: "live" is the latest pass or rollback up to that time —
    # worked out from the history, never stored in a separate file
    passing = []
    for event in load_history(path):
        counts = event["event"] in ("deploy_pass", "rollback")
        if counts and event["at"] <= timestamp:
            passing.append(event)
    if not passing:
        return None
    return passing[-1]["version_name"]

def get_live_version(path: str = "history.json") -> str | None:
    now = datetime.now(timezone.utc).isoformat()
    return get_live_version_at(now, path)

def deploy(version_name: str, threshold: float,
           versions_path: str = "versions.json",
           history_path: str = "history.json") -> DeployResult:
    record = get_version(version_name, versions_path)
    if record is None:
        raise VersionNotFoundError(f"No saved version named: {version_name}")

    result = run_eval_suite(record.config)
    passed = result["score"] >= threshold
    if passed:
        event_type = "deploy_pass"
    else:
```

```
        event_type = "deploy_fail"
    record_event(event_type, version_name, history_path)

    return DeployResult(
        version_name=version_name,
        passed=passed,
        score=result["score"],
        failing_cases=result["failed"],
    )
```

Story — `rollback.py`: a rollback must go back to the last version that *passed*, not just "whatever came before" — which could be a version the gate refused. **If not:** a rollback could quietly re-release a version that was already known to be broken.

```
# practice/build_task/rollback.py
from versioning import get_version
from deploy_gate import load_history, record_event

class NoPreviousVersionError(Exception):
    pass

def rollback_to_previous(history_path: str = "history.json",
                         versions_path: str = "versions.json"):
    passing = []
    for event in load_history(history_path):
        if event["event"] in ("deploy_pass", "rollback"):
            passing.append(event)

    if not passing:
        raise NoPreviousVersionError(
            "No version has ever passed the gate — nothing to roll back to."
        )

    current_live = passing[-1]["version_name"]
    previous = None
    # why: walk back through versions that PASSED — never to one that
    # failed the gate, and never to the one that's live right now
    for event in reversed(passing[:-1]):
        if event["version_name"] != current_live:
            previous = event["version_name"]
            break

    if previous is None:
        raise NoPreviousVersionError(
            f"No earlier passing version before {current_live}."
        )
```

```
    record_event("rollback", previous, history_path)
    return get_version(previous, versions_path)
```

`rollback_to_previous()` returns a full `VersionRecord`, matching the Build Task's stated interface, so the caller gets the restored version's `config` and `created_at` too.

```
# practice/build_task/main.py — proving it works
from versioning import save_version
from deploy_gate import deploy, get_live_version
from rollback import rollback_to_previous

save_version("v1", {"prompt": "Answer the customer's question."})
save_version("v2", {"prompt": "Answer politely and cite the source."})

deploy("v1", 0.5)
result = deploy("v2", 0.9)
print("v2 passed:", result.passed, "score:", result.score)
print("live:", get_live_version())

restored = rollback_to_previous()
print("rolled back to:", restored.version_name)
print("live:", get_live_version())
```

Expected output (`python main.py` , starting with no `versions.json` or `history.json`):

```
v2 passed: True score: 1.0
live: v2
rolled back to: v1
live: v1
```